Logics and Symbolic AI
Theorem Proving

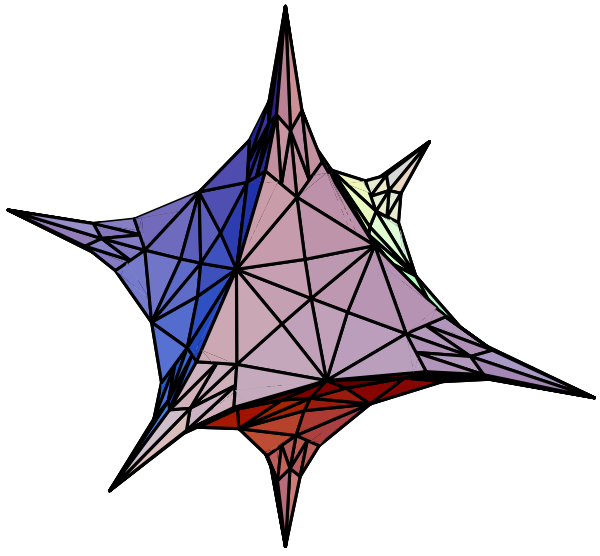
Thomas Bonald

2026 – 2027



INSTITUT
POLYTECHNIQUE
DE PARIS

Catalan's constant



$$G = 1 - \frac{1}{3^2} + \frac{1}{5^2} - \frac{1}{7^2} + \dots$$

Theorem: G is irrational

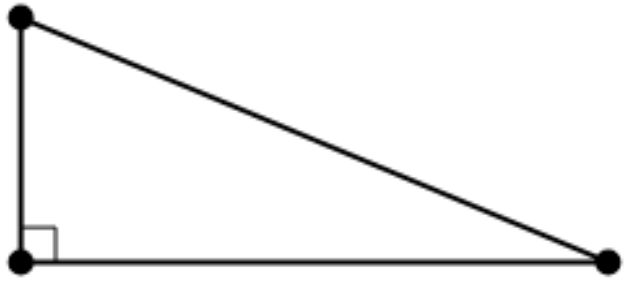
Proved by OpenAI on October 7, 2026

The AI Singularity in Mathematics

Since Antiquity, **mathematics** has been developed by **humans**



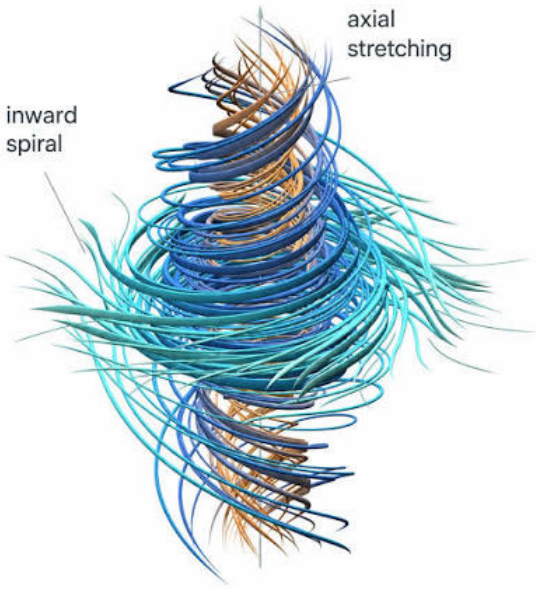
Euclide (300 BC)



The AI Singularity in Mathematics

Since Antiquity, **mathematics** has been developed by **humans**

Last summer, some **major** theorems are proved by AI, including the **Navier-Stockes** Millenium problem.



Source: OpenAI

The AI Singularity in Mathematics

Since Antiquity, **mathematics** has been developed by **humans**

Last summer, some **major** theorems are proved by AI, including the **Navier-Stockes** Millenium problem.

On October 7, 2026, **722** manuscripts are released by OpenAI, on

- number theory
- geometry
- analysis
- algebra
- probability

About **300** of them are formalized in LEAN.

Motivation

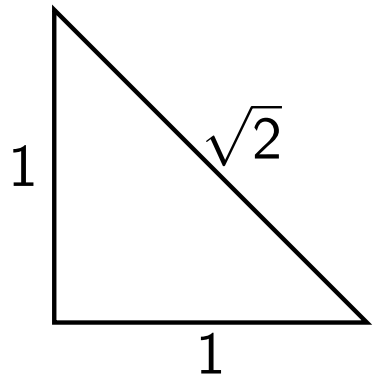
Theorem Proving is used in:

- **Mathematics**, to write formal proofs
  cf. Hilbert's program (1920s)
- **Computer Science** & **Engineering**, to check code
  cf. critical systems (aviation, protocols, hardware, security)

Main **tools**:

- LEAN (Microsoft)
- Isabelle (Cambridge / TUM)
- Coq / Rocq (Inria)

A proof



Theorem

$\sqrt{2}$ is irrational

Proof

- If $\sqrt{2}$ is rational, then $\sqrt{2} = \frac{a}{b}$ for some coprime numbers a, b
- Thus $a^2 = 2b^2$, so that a^2 is even, and a is even, say $a = 2k$
- Thus $2k^2 = b^2$, so that b^2 is even, and b is even
- This is a contradiction, as a and b are coprime

How to make this proof **formal** / **verifiable**?

The language of mathematics

All men are mortal.
Socrates is a man.
Therefore, Socrates is mortal

Aristotle (350 BC)

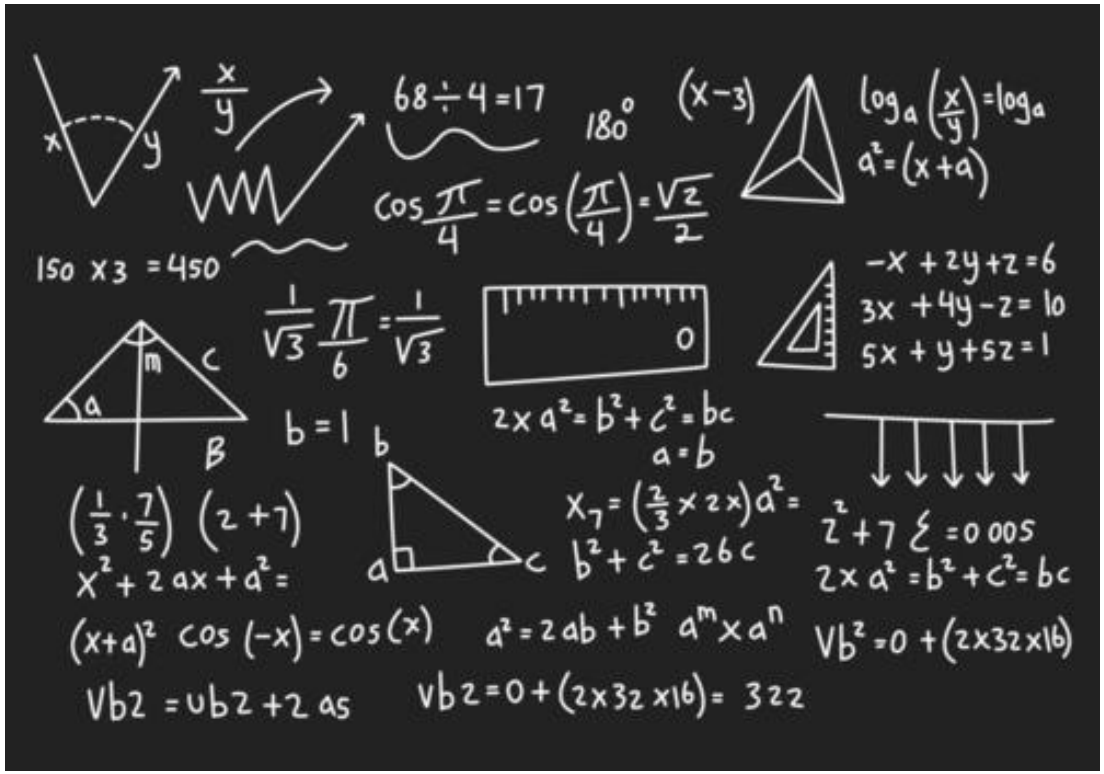
Proofs are based on **logic**:

$$A \to B \quad A \wedge B \quad A \vee B \quad \forall x, A \quad \exists x, A$$

In principle, they can be checked **automatically** based on some finite set of **axioms** and **inference rules**.

Hilbert's program (1920s)

Objective: To found mathematics on a finite set of **axioms**, ensuring the formal system is **consistent**[1], **complete**[2], and **decidable**[3].



[1] No contradiction.

[2] Either P or $\neg P$ has a proof.

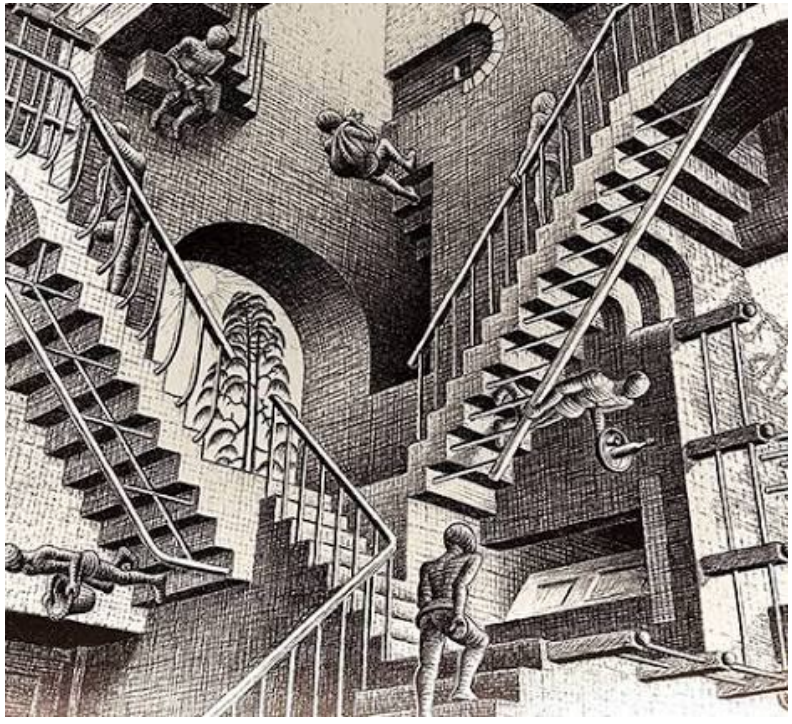
[3] There is a program deciding in finite time whether P or $\neg P$ has a proof.

Then came Gödel...

First Incompleteness Theorem (Gödel, 1931)

Any **consistent** formal system containing arithmetic is **incomplete**.

Consequence: Hilbert's program is **infeasible**!



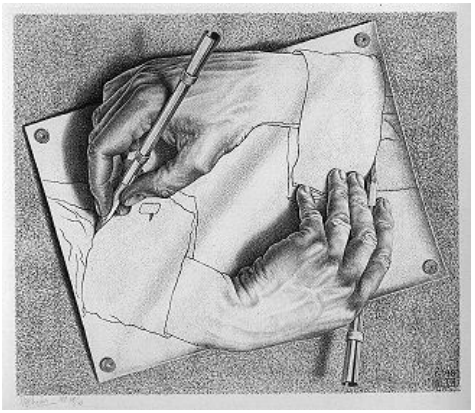
Relativity (Escher, 1953)

What remains?

We can still expect a **consistent** set of axioms (like ZF).

Second Incompleteness Theorem (Gödel, 1931)

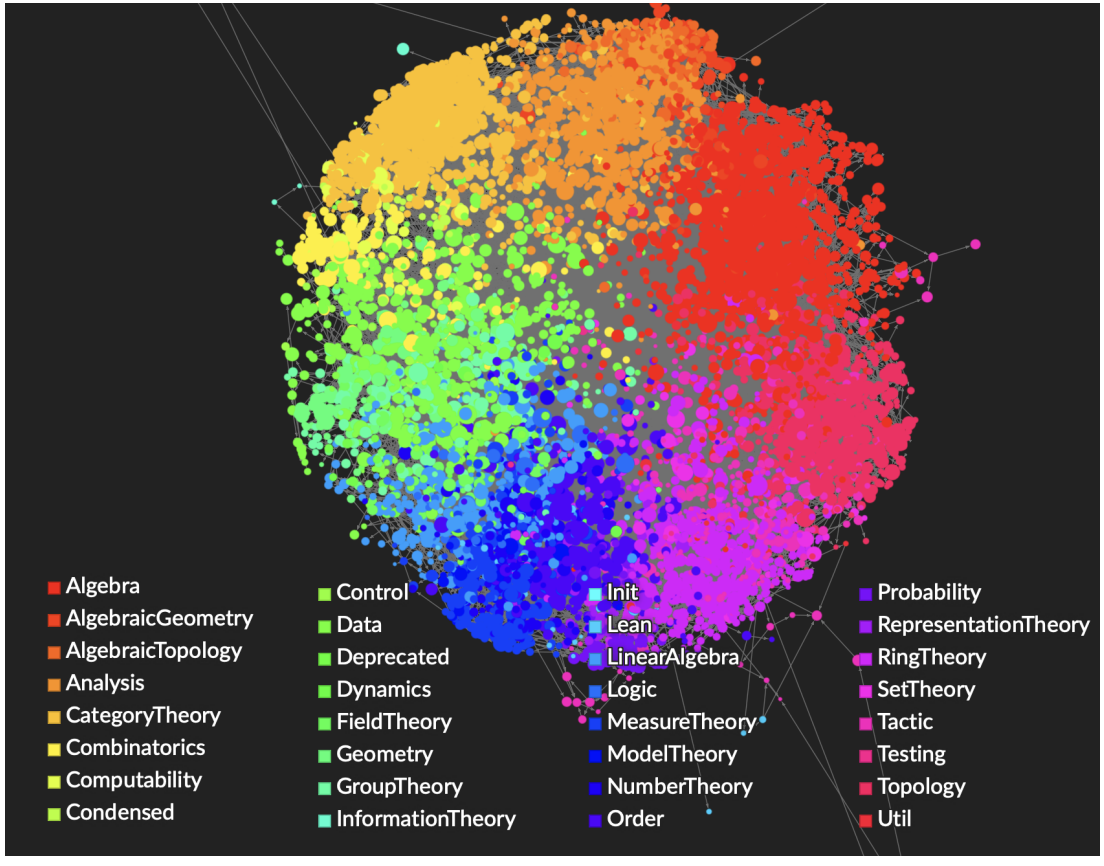
Any formal system containing arithmetic cannot prove its own **consistency**.



Drawing Hands (Escher, 1948)

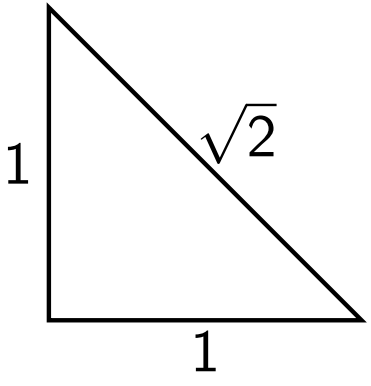
Mathlib

The largest **formal** mathematical library, coded in LEAN
$\sim$**275k theorems**... including Gödel's :-)
Less than 1% of all mathematics?



Import graph of Mathlib

Example




```
theorem irrational_sqrt_ratCast_iff_of_nonneg {q : ℚ} (hq : 0 ≤ q) :
    Irrational (√q) ↔ ¬IsSquare q := by
  refine Iff.not (?_ : Exists _ ↔ Exists _)
  constructor
  · rintro ⟨y, hy⟩
    refine ⟨y, Rat.cast_injective (α := ℝ) ?_⟩
    rw [Rat.cast_mul, hy, mul_self_sqrt (Rat.cast_nonneg.2 hq)]
  · rintro ⟨q', rfl⟩
    exact ⟨|q'|, mod_cast (sqrt_mul_self_eq_abs q').symm⟩

theorem irrational_sqrt_ratCast_iff {q : ℚ} :
    Irrational (√q) ↔ ¬IsSquare q ∧ 0 ≤ q := by
  obtain hq | hq := le_or_gt 0 q
  · simp_rw [irrational_sqrt_ratCast_iff_of_nonneg hq, and_iff_left hq]
  · rw [sqrt_eq_zero_of_nonpos (Rat.cast_nonpos.2 hq.le)]
    simp_rw [not_irrational_zero, false_iff, not_and, not_le, hq, implies_true]

theorem irrational_sqrt_intCast_iff_of_nonneg {z : ℤ} (hz : 0 ≤ z) :
    Irrational (√z) ↔ ¬IsSquare z := by
  rw [← Rat.isSquare_intCast_iff, ← irrational_sqrt_ratCast_iff_of_nonneg (mod_cast hz),
    Rat.cast_intCast]

theorem irrational_sqrt_intCast_iff {z : ℤ} :
    Irrational (√z) ↔ ¬IsSquare z ∧ 0 ≤ z := by
  rw [← Rat.cast_intCast, irrational_sqrt_ratCast_iff, Rat.isSquare_intCast_iff,
    Int.cast_nonneg_iff]

theorem irrational_sqrt_natCast_iff {n : ℕ} : Irrational (√n) ↔ ¬IsSquare n := by
  rw [← Rat.isSquare_natCast_iff, ← irrational_sqrt_ratCast_iff_of_nonneg n.cast_nonneg,
    Rat.cast_natCast]

theorem irrational_sqrt_ofNat_iff {n : ℕ} [n.AtLeastTwo] :
    Irrational √(ofNat(n)) ↔ ¬IsSquare ofNat(n) :=
  irrational_sqrt_natCast_iff

theorem Nat.Prime.irrational_sqrt {p : ℕ} (hp : Nat.Prime p) : Irrational (√p) :=
  irrational_sqrt_natCast_iff.mpr hp.not_isSquare
```

Morality

Proofs $\leftrightarrow$ Programs

Known as the **Curry-Howard correspondance**

Key ingredients:

- λ-calculus
- Type theory
- Propositional logic

λ-calculus

A (universal) model of **computation** based on **anonymous** functions:

$$\lambda x.y$$

Valid terms defined inductively by:

- **Variables**: $x, y, z, f, \ldots$
- **Abstraction**: $\lambda x.y$ creates the function $x \mapsto y$
- **Application**: fx applies the function f to x

Examples:

$$\lambda x.x$$

$$\lambda f.\lambda x.fx$$

$$\lambda f.\lambda x.f(f(fx))$$

$$\lambda x.\lambda y.\lambda z((xz)(yz))$$

Note: Variables are **not** typed!

Arithmetic

You can write arithmetic by encoding **numbers** and **operations**

Church encoding

Number	λ term
0	$\lambda f.\lambda x.x$
1	$\lambda f.\lambda x.fx$
2	$\lambda f.\lambda x.f(fx)$
n	$\lambda f.\lambda x.f^n x$

Operation	λ term
$n+1$	$\lambda n.\lambda f.\lambda x.f(nfx)$
$m+n$	$\lambda m.\lambda n.\lambda f.\lambda x.mf(nfx)$
mn	$\lambda m.\lambda n.\lambda f.\lambda x.m(nf)x$
$\vdots$	$\vdots$

A (modest) result

Proposition

$$1 + 2 = 3$$

Proof

$$\underbrace{\lambda n.\lambda m.\lambda f.\lambda x.nf(mfx)}_{+}\ \underbrace{\lambda f.\lambda x.fx}_{1}\ \underbrace{\lambda f.\lambda x.f(fx)}_{2}$$

$$= \lambda m.\lambda f.\lambda x.f(mfx)\ \underbrace{\lambda f.\lambda x.f(fx)}_{2}$$

$$= \underbrace{\lambda f.\lambda x.f(f(fx))}_{3}$$

Logic

You can write logic by encoding **booleans** and **connectors**

Church encoding

Boolean	λ term
True	$\lambda a.\lambda b.a$
False	$\lambda a.\lambda b.b$

Connector	λ term
$\neg$	$\lambda p.p$.False True
$\wedge$	$\lambda p.\lambda q.pqp$
$\vee$	$\lambda p.\lambda q.ppq$
$\vdots$	$\vdots$

Normalization

Expressions of the form $(\lambda x.y)z$ can be **reduced**:

$$(\lambda x.xx)y \rightarrow yy$$
$$(\lambda x.\lambda y.x)z \rightarrow \lambda y.z$$

A λ-term that cannot be reduced is in **normal form**:

$$\lambda f.\lambda x.fx$$

$$\lambda f.\lambda x.f(f(fx))$$

$$\lambda x.\lambda y.\lambda z((xz)(yz))$$

Strong Normalization

A λ-term in **strongly normalizing** if **any** possible sequence of reductions terminates in **finite time** to the normal form.

Counter-example: $(\lambda x.xx)(\lambda x.xx)$

Typing

Given types σ, τ, define the **function type**:

$$\sigma \to \tau$$

Terms are typed inductively by:

- **Variables**: $x : \sigma, y : \tau, \ldots$
- **Abstraction**: $\lambda x.y$ with $x : \sigma$ and $y : \tau$ has type $\sigma \to \tau$
- **Application**: fx, defined only if $f : \sigma \to \tau$ and $x : \sigma$, has type τ

This is called **simply typed** λ-calculus.

Strong Normalizing Theorem (Curry, 1930s)

Simply typed λ-calculus is **strongly normalizing**.

Note: The term $(\lambda x.xx)(\lambda x.xx)$ cannot be typed!

Propositional logic

Formulas are defined inductively as:

- Propositional variables $A, B, \ldots$
- $A \to B$, $A \wedge B$, $A \vee B$
- $\top$ (true) and $\bot$ (false)

The **rules** of **natural deduction** are:

	Introduction	**Elimination**
$\to$	$\dfrac{\Gamma, A \vdash B}{\Gamma \vdash A \to B}$	$\dfrac{\Gamma \vdash A \to B \quad \Gamma \vdash A}{\Gamma \vdash B}$
$\wedge$	$\dfrac{\Gamma \vdash A \quad \Gamma \vdash B}{\Gamma \vdash A \wedge B}$	$\dfrac{\Gamma \vdash A \wedge B}{\Gamma \vdash A} \quad \dfrac{\Gamma \vdash A \wedge B}{\Gamma \vdash B}$
$\vee$	$\dfrac{\Gamma \vdash A}{\Gamma \vdash A \vee B} \quad \dfrac{\Gamma \vdash B}{\Gamma \vdash A \vee B}$	$\dfrac{\Gamma \vdash A \vee B \quad \Gamma, A \vdash C \quad \Gamma, B \vdash C}{\Gamma \vdash C}$
$\top$	$\dfrac{}{\Gamma \vdash \top}$	
$\bot$		$\dfrac{\Gamma \vdash \bot}{\Gamma \vdash A}$

Curry-Howard correspondance

Proofs $\leftrightarrow$ Programs

Natural deduction	**Simply typed λ-calculus**
$\dfrac{\Gamma, A \vdash B}{\Gamma \vdash A \to B}\,\text{Intro} \to$	$\dfrac{\Gamma, x : \sigma \vdash y : \tau}{\Gamma \vdash \lambda(x : \sigma).y : \sigma \to \tau}\,\text{Abs}$
$\dfrac{\Gamma \vdash A \to B \quad \Gamma \vdash A}{\Gamma \vdash B}\,\text{Elim} \to$	$\dfrac{\Gamma \vdash f : \sigma \to \tau \quad \Gamma \vdash x : \sigma}{\Gamma \vdash fx : \tau}\,\text{App}$

Extensions

Other connectors correspond to **extended** types:

Formula	**Type**
$A \to B$	$\sigma \to \tau$
$A \wedge B$	$\sigma \times \tau$
$A \vee B$	$\sigma + \tau$
$\top$	1
$\bot$	0

The corresponding typed λ-calculus remains **strongly normalizing**.

Summary

Proofs $\leftrightarrow$ Programs

Logic	**Computation**
formula	type
proof	term
$\rightarrow$	function type
$\wedge$	product type
$\vee$	sum type
$\top$	unit type
$\bot$	empty type
$\forall$	dependent product type
$\exists$	dependent sum type

Types in LEAN

Any expression in LEAN has a **type**.

Type	Example
Primitive	`Bool`
Function	`Type → Type`
Product	`Type × Type`
Sum	`Type ⊕ Type`
Inductive	`Nat`

Example

```
inductive Nat where
  | zero : Nat
  | succ : Nat → Nat
```

☛ LEAN relies on **dependent type theory**.

Tactics in LEAN

LEAN provides **tactics** for **interactive** theorem proving.

Tactic	Examples
Basic	`rfl, rw, intro, exact, apply`
Induction	`induction`
Contradiction	`by_contra`
Advanced	`linarith, ring, simp`

Example

```
example (a b : ℝ) (h : a * b = 1) : a * (b + 1) = a + 1 := by
  rw [mul_add, h, mul_one, add_comm]
```

☛ cf. Lab

Constructive proofs

Theorem

There exist irrational numbers a, b such that a^b is rational.

Proof

Let $x = \sqrt{2}^{\sqrt{2}}$

- If x is rational, take $a = b = \sqrt{2}$.
- Otherwise, take $a = x, b = \sqrt{2}$ as $a^b = x^{\sqrt{2}} = \sqrt{2}^2 = 2$.

This proof is **not** constructive!
It relies on the following axiom of **classical logic**:

Law of the Excluded Middle

$$P \vee \neg P$$

AI & Theorem proving

AI can write theorems...

1. Can we **trust** the proofs?
   Yes, if checked by a formal language
   (but the translation is not easy)
2. Can we **understand** them?
   No (most often)
   (but AI might provide explanations)
3. Do we still need **mathematicians**?
   Yes (almost surely)
   (to explore the ocean of theorems generated by AI)

